

PONTIFÍCIA UNIVERSIDADE CATÓLICA DE GOIÁS
BIG DATA E INTELIGÊNCIA ARTIFICIAL



PROJETO INTEGRADOR IV – A
SECRETARIA DE ESTADO DA SAÚDE DE GOIÁS
**CONSTRUÇÃO DE PIPELINE PARA PROCESSAMENTO DISTRIBUIDO DE
DADOS FINANCEIROS DA SES-GO**

Guilherme Avelino Gomes
Ivânio José da Rocha Júnior

GOIÂNIA
2026

SUMÁRIO

Introdução	3
2. Desenvolvimento	3
2.1 Contextualização	3
2.2 Arquitetura em Nuvem e Processamento Paralelo e Distribuído.....	4
3. Conclusão	5
4. Referências Bibliográficas.....	6
Anexos	7
Anexo I – Estrutura de um dos arquivos de carga.....	7
Anexo II – Arquitetura da Solução Proposta	8
Anexo III – Data Lake e Databricks Criados.....	9
Anexo IV – Estrutura dos dados do Data Lake Bronze	10
Anexo V – Estrutura dos dados do Data Lake Silver	10
Anexo VI – Databricks.....	11
Anexo VII – Painel Estatísticas de Dotação e Empenhos – Power BI.....	12
Anexo VIII – Código Fonte do ETL e Processamento	13

Introdução

Este projeto nasceu da necessidade de monitorar os dados do Sistema de Execução Financeira da Secretaria de Estado da Saúde de Goiás (SES-GO). Seu propósito é otimizar o processamento de informações orçamentárias, facilitando o acompanhamento gerencial por parte dos gestores da pasta.

O objetivo é disponibilizar uma ferramenta que permita a visualização dos saldos em tempo real, com acurácia e garantindo que a gestão financeira esteja plenamente alinhada às diretrizes estabelecidas.

2. Desenvolvimento

2.1 Contextualização

A execução orçamentária do Estado de Goiás é regida pela **Lei Federal nº 4.320/64**, integrando especificidades locais como o sistema **SIOF/SIAFIC** e as diretrizes da **Secretaria da Economia**. No âmbito da Secretaria de Estado da Saúde (SES-GO), essa dinâmica demanda máxima eficiência no processamento de dados para garantir a continuidade dos serviços públicos.

Nesse cenário, a automação da carga de dados provenientes da Secretaria da Economia (via arquivos .txt) é uma necessidade crítica. O processamento é diário e de alta complexidade, pois cada instrumento financeiro pode gerar múltiplos desdobramentos e atualizações estatísticas.

O rito operacional da SES-GO observa a seguinte cronologia legal e técnica:

1. **Programação e Dotação:** Estabelecimento do cronograma de desembolso mensal. O orçamento é liberado aos órgãos via cotas (Dotações), habilitando o uso dos recursos.
2. **Empenho:** Reserva do crédito orçamentário para um compromisso específico (contrato ou compra), formalizando a obrigação de pagamento.
3. **Liquidação:** Verificação e ateste da prestação do serviço ou entrega do produto. Consolida o direito do credor, tornando o gasto "líquido e certo".
4. **Pagamento:** Fase final com a emissão da Ordem de Pagamento (OB) e a efetiva transferência financeira ao fornecedor.

A cada etapa, são geradas estatísticas para validação de saldos e conformidade financeira. Consequentemente, uma única operação de compra pode originar diversos registros correlacionados. Atualmente, o ecossistema de dados apresenta os seguintes desafios:

- **Histórico e Volume:** Base de dados abrangendo o período de **2003 a 2026**.
- **Carga Diária:** Aproximadamente **177 arquivos diários**, totalizando cerca de **1,5 GB**.

- **Escalabilidade:** O volume de dados apresenta crescimento linear, tornando o tratamento manual técnica e operacionalmente inviável.
- **Complexidade Estrutural:** Cada linha de arquivo representa um instrumento financeiro com múltiplos atributos. A separação manual e o processamento tempestivo das estatísticas são impraticáveis sem suporte tecnológico (ver Anexo I).

A Secretária da Economia disponibiliza diariamente os arquivos dos dados, segundo um layout específico e disponibilizado junto com este trabalho. A solução não deve ser restrita à SES-GO, pois outras Secretarias têm a mesma necessidade de processamento das informações.

2.2 Arquitetura em Nuvem e Processamento Paralelo e Distribuído

A solução não deve ser restrita à SES-GO. A arquitetura deve ser modular para permitir a migração ou o uso simultâneo por outras secretarias que compartilham a estrutura financeira estadual. A arquitetura da solução encontra-se no Anexo II.

Adotamos a solução de PaaS do Microsoft Azure, pela sua disponibilidade na infraestrutura das Secretarias Estaduais e para suportar o volume histórico e crescente de informações. Implementamos um modelo de processamento distribuído utilizando o Azure Databricks integrado ao Azure Data Lake Storage (ADLS).

Com base no layout de arquivos disponibilizados (arquivo Layout.pdf), a solução focou na construção de um pipeline de **ETL (Extract, Transform, Load)** robusto, capaz de realizar a tipagem de dados no momento da ingestão e o uso do processamento em memória "schema on read".

A ingestão ocorre na camada Bronze em formato bruto (.txt) e o processamento é realizado com Apache Spark e usando a API PySpark. Como são arquivos desde 2003 e o volume total de linhas é alto (passou de 1,4 milhão de linhas), o Spark foi escolhido por ter uma performance melhor neste cenário. Foram utilizadas as seguintes técnicas identificadas no desenvolvimento:

1. **Fatiamento de Dados (Parsing):** Como os arquivos seguem um layout de posição fixa, utilizamos a função substring do Spark para decompor as "tripas" de texto em colunas estruturadas (ex: Empenho, Valor, Data, Credor), garantindo precisão na tipagem dos dados. Fatiar uma "tripa" de texto de posição fixa (como o seu substr(1, 2)) em um loop Python comum seria muito lento. Com o Spark, isso é distribuído entre todos os núcleos do processador do cluster simultaneamente.
2. **Otimização em Memória (Caching):** Para garantir celeridade, implementamos o comando .cache() logo após a leitura dos dados brutos. É uma função pura do Spark que "prende" os dados na RAM, permitindo que as transformações seguintes (o fatiamento das strings) sejam instantâneas. Isso fixa os mais de 1,4 milhão de registros (DUEOF e Liquidações) na memória RAM do cluster, eliminando gargalos de leitura de disco durante as transformações.
3. **Tratamento de Evolução Histórica:** O código foi preparado para lidar com diferentes layouts (DUEOF, Liquidação Tipo D e M), tratando as divergências de

formato no momento do fatiamento, o que permite consolidar dados de 2003 a 2024 em uma visão única. Os cálculos dos saldos das dotações, empenhos, liquidações e ordens de pagamento foram realizados nesta camada, considerando os tipos de documentos e seus efeitos financeiros (somar ou subtrair valor).

Após o tratamento, os dados foram consolidados em tabela de negócio com os saldos dos instrumentos financeiros (Dotação e Painel de Empenhos) e exportados para a camada **Silver** (Data Lake) em formato CSV estruturado. Posteriormente transferidos para o Power BI e gerado o dashboard final (Anexo XX).

Os principais ganhos desta abordagem são:

1. **Custo-Eficiência:** O armazenamento em Data Lake reduz drasticamente os custos em comparação a bancos SQL tradicionais;
2. **Performance:** A base de dados para ferramentas como o **Power BI** torna-se extremamente leve, pois a ferramenta de visualização consome arquivos já processados e otimizados pelo cluster Spark e
3. **Modularidade:** O mesmo dado bruto pode ser "fatiado" de formas diferentes para atender à SES-GO ou qualquer outra Secretaria, bastando ajustar a lógica de extração no Databricks.

Os arquivos do projeto estão disponíveis no seguinte git hub:
https://github.com/ivaniojr/projeto_integrador_IV_Guilherme_Ivanio.git

3. Conclusão

Este projeto consolidou uma solução robusta para o monitoramento da execução financeira da SES-GO, superando os gargalos do processamento manual de dados. A arquitetura em nuvem, baseada em **Azure Databricks** e **Apache Spark**, permitiu transformar mais de 1,4 milhão de registros históricos em inteligência estratégica com alta performance e baixo custo.

A estratégia de *schema on read* via **PySpark** foi determinante para o fatiamento preciso de arquivos de posição fixa, garantindo a integridade dos saldos e a conformidade orçamentária em um ambiente distribuído. Em suma, a entrega do dashboard em **Power BI** automatiza o fluxo de ETL e assegura uma tomada de decisão pautada pela transparência e acurácia.

Como evolução, propõe-se a implementação de um modelo **multiórgão** centralizado no mesmo cluster. Essa abordagem permitiria que diversas Secretarias utilizassem a mesma infraestrutura de processamento, eliminando a necessidade de instâncias individuais e otimizando os recursos tecnológicos do Estado.

4. Referências Bibliográficas

TANENBAUM, A. S.; VAN STEEN, M. Sistemas Distribuídos: Princípios e Paradigmas. Pearson, 2017.

WHITE, T. Hadoop: The Definitive Guide. O'Reilly, 2015.

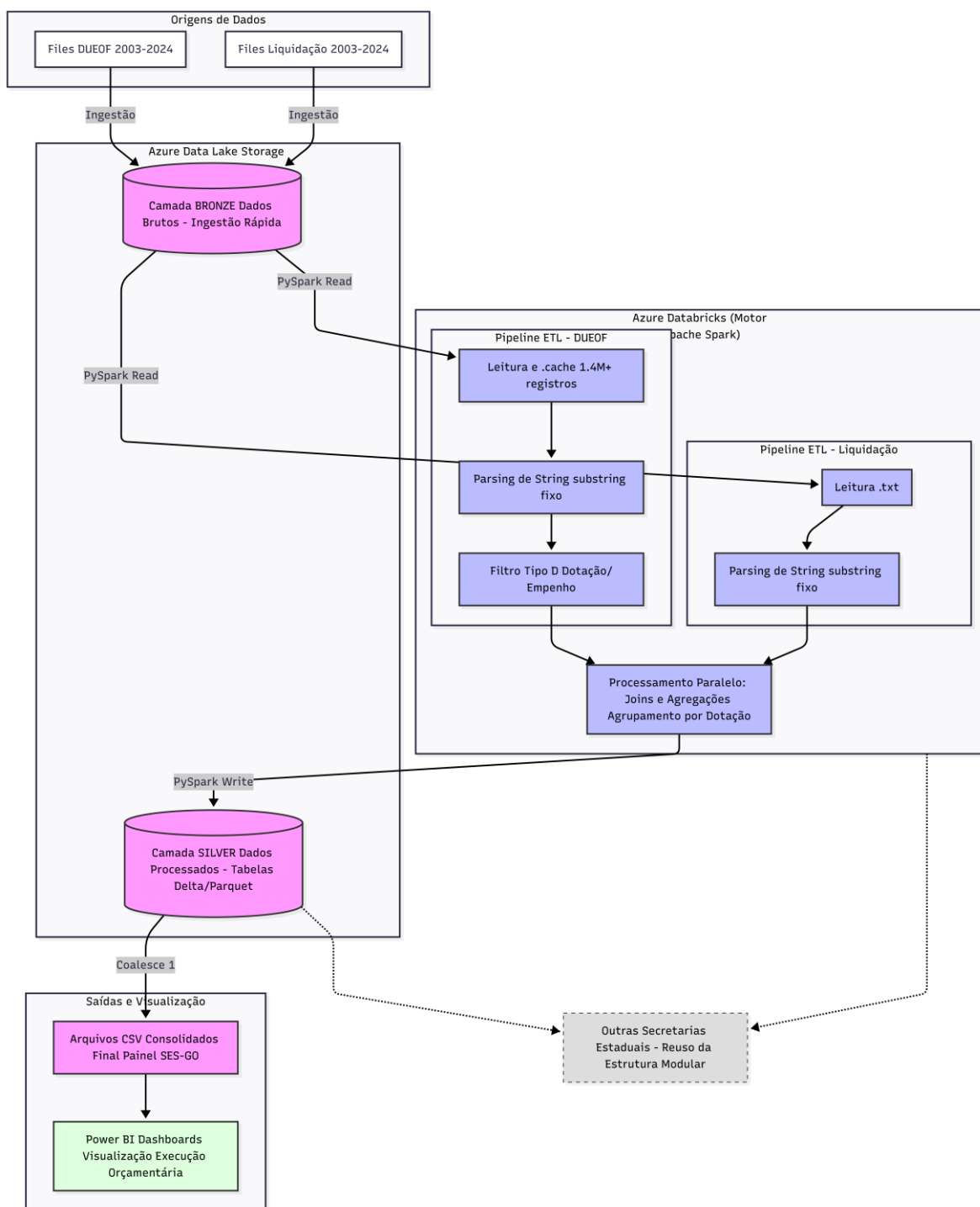
KLEPPMANN, M. Designing Data-Intensive Applications. O'Reilly, 2017.

MARZ, N.; WARREN, J. Big Data: Principles and Best Practices of Scalable Real-Time Data Systems. Manning, 2015.

Anexo I – Estrutura de um dos arquivos de carga.

7

Anexo II – Arquitetura da Solução Proposta



Anexo III – Data Lake e Databricks Criados.

Microsoft Azure

Página inicial > Resource Manager

Resource Manager | Tudo

Identificar problemas de desempenho nos recursos | Sugerir serviços relacionados a esta lista | Comparar métricas entre esses recursos

Pesquisar recursos, serviços e documentos (G+J) | Copilot

Nome 1 Tipo Grupo de Recursos Localização Assinatura

Nome	Tipo	Grupo de Recursos	Localização	Assinatura
datalakeivangui	Conta de armazenamento	Projeto-Integrador	East US	Azure subscription 1
dbmanagesidentity	Identidade Gerenciada	databricks-rg-guilherme-novo	West US 2	Azure subscription 1
dbstorage54p6wvnrqps	Conta de armazenamento	databricks-rg-guilherme-novo	West US 2	Azure subscription 1
GuilhermeVianciWorkspace	Serviço do Azure Databricks	Projeto-Integrador	West US 2	Azure subscription 1
nat-gateway	Gateway da NAT	databricks-rg-guilherme-novo	West US 2	Azure subscription 1
nat-gw-public-ip	Endereço do IP público	databricks-rg-guilherme-novo	West US 2	Azure subscription 1
NetworkWatcher_eastus	Observador de Rede	NetworkWatcherRG	East US	Azure subscription 1
NetworkWatcher_westus2	Observador de Rede	NetworkWatcherRG	West US 2	Azure subscription 1
unity-catalog-access-connector	Conector de acesso para Azure...	databricks-rg-guilherme-novo	West US 2	Azure subscription 1
workers-rg	Grupo de segurança de rede	databricks-rg-guilherme-novo	West US 2	Azure subscription 1
workers-vnet	Rede virtual	databricks-rg-guilherme-novo	West US 2	Azure subscription 1

Microsoft Azure

Página inicial > Resource Manager | Todos os recursos > datalakeivangui

Contêineres

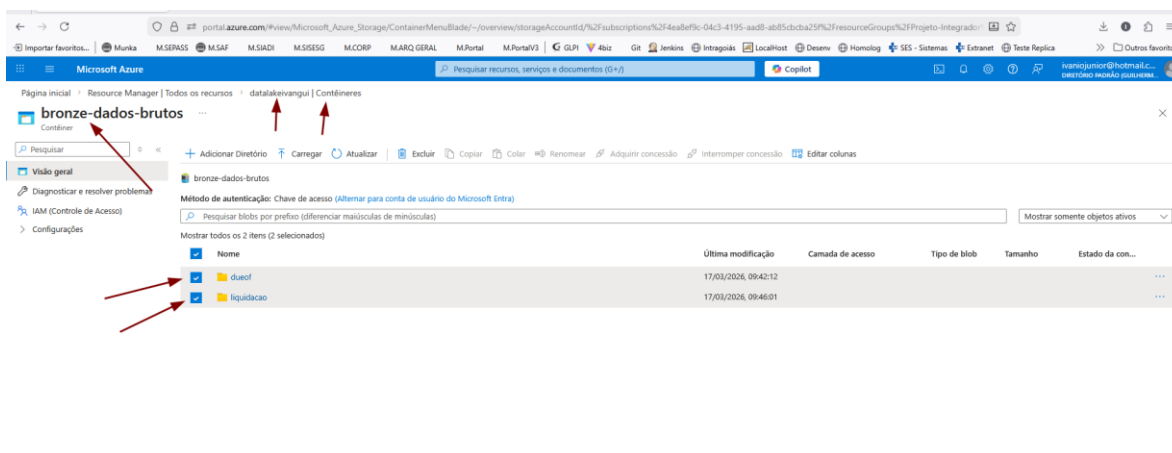
Adicionar contêiner | Carregar | Atualizar | Excluir | Alterar o nível de acesso | Restaurar contêineres | Editar colunas

Pesquisar contêineres por prefixo

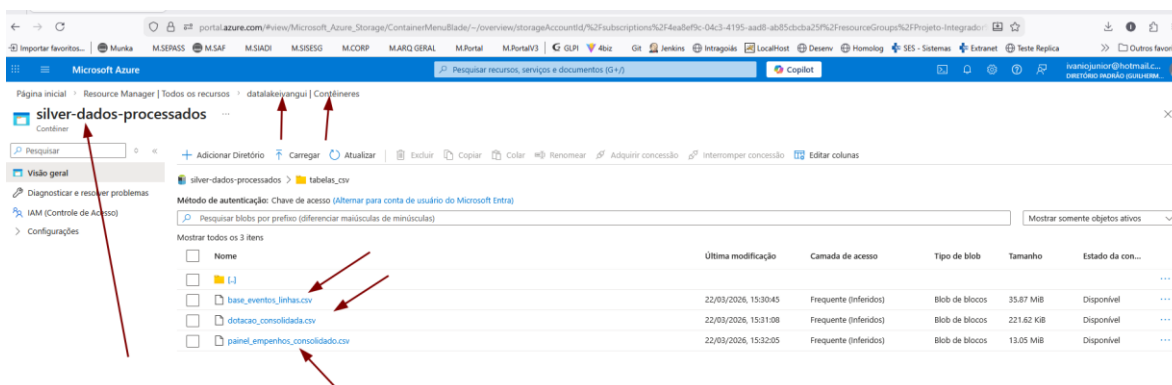
Mostrar todos os 3 itens

Nome	Última modificação	Nível de acesso anônimo	Estado da con...
slogs	17/03/2026, 00:33:41	Privado	Disponível
bronze-dados-brutos	17/03/2026, 00:35:58	Privado	Disponível
silver-dados-processados	19/03/2026, 18:16:50	Privado	Disponível

Anexo IV – Estrutura dos dados do Data Lake Bronze



Anexo V – Estrutura dos dados do Data Lake Silver

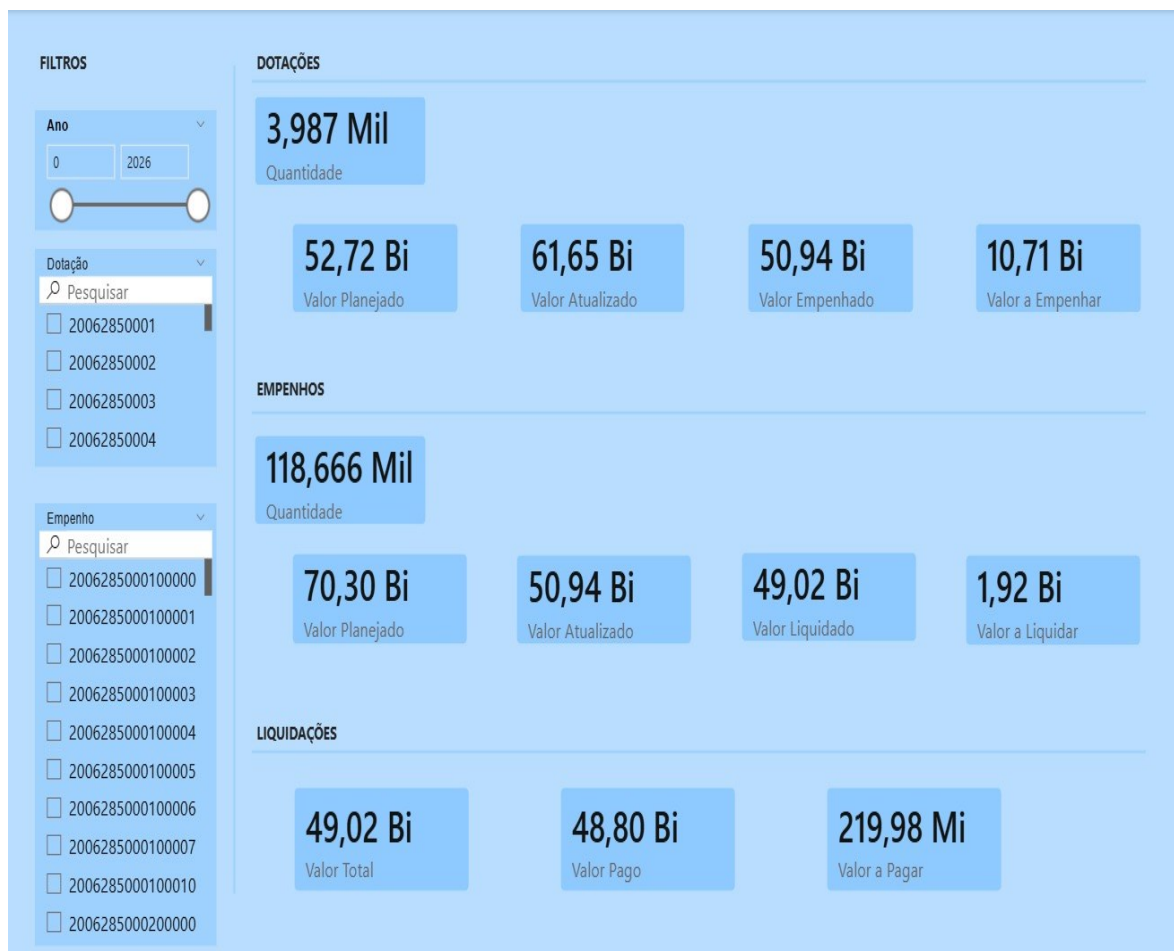


Anexo VI – Databricks

Microsoft Azure portal showing the configuration page for the 'GuilhermelvanioWorkSpace' Databricks workspace. The page displays various settings under 'Fundamentos' (Basics), including Status (Active), Group of resources (Projeto Integrado), Local (West US 2), Signature (Azure subscription), and ID of the signature (4eaefffc-04c3-4195-aad8-ab85cbcb25f). A 'Marcar' (Tag) field is also visible. On the right, there are links for 'Iniciar o Workspace' (Start the Workspace) and 'Atualizar para Premium' (Upgrade to Premium). The page also shows the 'Tipo de workspace' (Hybrid), 'Grupo de Recursos Gere...' (databricks-rg-guilherme-some), 'URL' (https://adb-7405605732276415.15.azure.databricks.net), 'Tipo de Preço' (Standard (Apache Spark, Sequoia com o Microsoft Entra ID / Clique para alterar)), and 'Habilitar Nenhum IP Públ...' (Sim).

Databricks workspace interface showing a Python script in a notebook. The script is titled 'processamento-projeto' and contains code for connecting to Azure storage, reading data from 'dados-brutos' and 'liquidações', and displaying the results. The code includes comments in Portuguese and uses Spark and Databricks APIs. The interface shows the 'processamento-projeto' notebook open in the 'GuilhermelvanioWorkSpace'.

Anexo VII – Painel Estatísticas de Dotação e Empenhos – Power BI



Anexo VIII – Código Fonte do ETL e Processamento

Este é o código fonte em python do ETL e do Processamento dos dados.

```
# Databricks notebook source
```

```
# CONEXÃO E LEITURA DOS DADOS BRUTOS EM TXT
```

```
# 1. Credenciais
```

```
storage_account_name = "datalakeivangui"
```

```
storage_account_key =
```

```
"uptABhZb3+kqJP0ac8jkdKnyKsjJkmKEfE9XjK0poagNWb85gzkwQYl7T+TXByh8ZYpPFa9dgP  
yJ+ASt0G1gQA=="
```

```
container_name = "bronze-dados-brutos"
```

```
# 2. Caminhos para as duas pastas específicas
```

```
path_dueof =
```

```
f"wasbs://{container_name}@{storage_account_name}.blob.core.windows.net/dueof/*.txt"
```

```
path_liquidacao =
```

```
f"wasbs://{container_name}@{storage_account_name}.blob.core.windows.net/liquidacao/*.t  
xt"
```

```
# 3. Leitura dos Empenhos (dueof)
```

```
df_dueof_bruto = spark.read \
```

```
    .option(f"fs.azure.account.key.{storage_account_name}.blob.core.windows.net",  
storage_account_key) \
```

```
    .text(path_dueof)
```

```
# 4. Leitura das Liquidações
```

```
df_liq_bruto = spark.read \
```

```
    .option(f"fs.azure.account.key.{storage_account_name}.blob.core.windows.net",  
storage_account_key) \
```

```

.text(path_liquidacao)

# Testando se os dois funcionam
print("Amostra de Empenhos (dueof):")
display(df_dueof_bruto.limit(5))

print("Amostra de Liquidações:")
display(df_liq_bruto.limit(5))

# COMMAND -----

# --- OTIMIZAÇÃO: MEMORY CACHE (SEM GRAVAÇÃO EM DISCO) ---

print("Carregando e fixando dados na memória do cluster...")

# 1. O comando .cache() avisa ao Spark: "Leia o TXT uma vez e guarde na RAM"
df_dueof_pronto = df_dueof_bruto.cache()
df_liq_pronto = df_liq_bruto.cache()

# 2. O comando .count() é uma "Ação" que força o Spark a ler os dados AGORA
# Sem isso, o cache só aconteceria na próxima vez que você desse um display
print(f"DUEOF carregado: {df_dueof_pronto.count()} linhas.")
print(f"Liquidações carregadas: {df_liq_pronto.count()} linhas.")

# 3. Mantendo a compatibilidade com o resto do seu código
# df_dueof = df_dueof_pronto

```

```

# df_liq = df_liq_pronto

print("Sucesso! Os dados estão presos na memória e prontos para o fatiamento.")

# COMMAND -----

from pyspark.sql.functions import col, trim, lit, concat, sum, when, substring, to_date, lpad,
min

from pyspark.sql.types import DecimalType

from pyspark.sql import Window

# COMMAND -----

#FATIA A TRIPA E CRIA UM DATAFRAME COM TODOS DADOS DISPONIVEIS
df_dueof_estruturado = df_dueof_pronto.select(
    col("value").substr(1, 2).alias("TIPO_DUEOF"),
    col("value").substr(3, 23).alias("CHAVE"),
    col("value").substr(3, 16).alias("NUMEMPENHO"),
    col("value").substr(26, 8).alias("DATA"),
    col("value").substr(34, 15).alias("NUMPROCESSO"),
    (col("value").substr(49, 14).cast(DecimalType(18, 2)) / 100).alias("VALOR"),
    col("value").substr(63, 14).alias("NUMR_CPF_CNPJ"),
    col("value").substr(77, 5).alias("FORMALIDADE_FINALIDADE"),
    col("value").substr(82, 11).alias("CONTA_BANCO_DEBITO"),
    col("value").substr(93, 11).alias("CONTA_BANCO_CREDITO"),
    col("value").substr(104, 2).alias("GRUPO_DESPESA"),
    col("value").substr(106, 60).alias("NOME_CREDOR"),

```

```

col("value").substr(166, 9).alias("CODIGO_BANCO_DEBITO"),
col("value").substr(175, 9).alias("CODIGO_BANCO_CREDITO"),
col("value").substr(184, 3).alias("PRAZO_APLICACAO"),
col("value").substr(187, 2).alias("STATUS_DOCUMENTO"),
col("value").substr(189, 8).alias("DATA_DO_STATUS"),
col("value").substr(197, 8).alias("NATUREZA"),
col("value").substr(205, 13).alias("CODIGO_DO_PATRIMONIO"),
col("value").substr(218, 1).alias("TIPO_EMPENHO"),
col("value").substr(219, 10).alias("NUMERO_DECRETO"),
col("value").substr(229, 8).alias("DATA_LEI"),
col("value").substr(237, 7).alias("NUMERO_LEI"),
col("value").substr(244, 2).alias("FUNCAO"),
col("value").substr(246, 3).alias("SUB_FUNCAO"),
col("value").substr(249, 4).alias("PROGRAMA"),
col("value").substr(253, 4).alias("ACAO"),
col("value").substr(257, 8).alias("FONTE"),
col("value").substr(265, 12).alias("RECEITA_DEBITO"),
col("value").substr(277, 12).alias("RECEITA_CREDITO"),
col("value").substr(289, 13).alias("NUMERO_PDF"),
col("value").substr(302, 2).alias("MODALIDADE_APLICACAO"),
col("value").substr(304, 11).alias("FILLER")
)

# Visualização do resultado final

display(df_dueof_estruturado.limit(13))

# COMMAND -----

```



```
#FATIA A TRIPA E CRIA DOIS DATAFRAMES - TXT LIQ TEM 2 LAYOUTS
```

```
df_raw_D = df_liq_pronto.filter(col("value").substr(1, 1) == "D")
```

```
df_raw_M = df_liq_pronto.filter(col("value").substr(1, 1) == "M")
```

```
# ESTRUTURAÇÃO DO REGISTRO 'D'
```

```
df_liq_D = df_raw_D.select(  
    col("value").substr(1, 1).alias("TIPO_DUEOF"),  
    col("value").substr(2, 16).alias("NUMEMPENHO"),  
    (col("value").substr(18, 17).cast(DecimalType(19, 2)) / 100).alias("VALOR"),  
    col("value").substr(35, 8).alias("DATA"),  
    col("value").substr(43, 2).alias("TIPODOCUMENTO"),  
    col("value").substr(45, 8).alias("DATADOCUMENTO"),  
    col("value").substr(53, 20).alias("NUMERODOCUMENTO"),  
    trim(col("value").substr(73, 120)).alias("DESCRICAO"),  
    col("value").substr(193, 9).alias("NUMEROSERIE"),  
    col("value").substr(202, 5).alias("MODELONOTA"),  
    col("value").substr(207, 8).alias("DATAVALIDADENOTA"),  
    col("value").substr(215, 8).alias("DATA DE REFENCIA"),  
    col("value").substr(223, 20).alias("NUMERO_FILA"),  
    trim(col("value").substr(243, 150)).alias("DESC_FILA")  
)
```

```
# ESTRUTURAÇÃO DO REGISTRO 'M'
```

```
df_liq_M = df_raw_M.select(  
    col("value").substr(1, 1).alias("TIPO_DUEOF"),
```

```

col("value").substr(2, 16).alias("NUMEMPENHO"),
col("value").substr(18, 20).alias("NUMERODOCUMENTO"),
col("value").substr(38, 4).alias("SEQMOVLIQ"),
col("value").substr(42, 1).alias("TIPOMOVIMENTO"),
(col("value").substr(43, 17).cast(DecimalType(19, 2)) / 100).alias("VALOR"),
col("value").substr(60, 8).alias("DATA"),
col("value").substr(68, 19).alias("OPREFERENCIA"),
col("value").substr(87, 22).alias("MOVOPREFERENCIA"),
col("value").substr(109, 1).alias("MOVOPTIPO"),
col("value").substr(110, 105).alias("BRANCOS")
)

```

Visualização para conferência

```
print("Amostra de Registros Tipo D:")
```

```
display(df_liq_D.limit(5))
```

```
print("Amostra de Registros Tipo M:")
```

```
display(df_liq_M.limit(5))
```

COMMAND -----

#AJUSTE DOS DATAFRAMES PARA MESMO LAYOUT E JUNÇÃO (DUEOF, LIQ_D, LIQ_M)

Lista de tipos para o DUEOF

```
tipos_dueof_selecionados = [
```

```
"01", "02", "03", "04", "05", "06", "07", "08", "12", "13", "25",  
"26", "27", "30", "31", "32", "33", "34", "35", "36", "37", "38", "46", "45"  
]
```

1. Preparação do DUEOF (Filtrando os tipos específicos)

```
df_dueof_ready = df_dueof_estruturado.select(  
    col("TIPO_DUEOF"),  
    col("NUMEMPENHO"),  
    col("VALOR"),  
    col("DATA"),  
    col("NUMPROCESSO")  
) .filter(col("TIPO_DUEOF").isin(tipos_dueof_selecionados))
```

2. Preparação do Liq_D (Todas as instâncias)

```
df_liq_D_ready = df_liq_D.select(  
    col("TIPO_DUEOF"),  
    col("NUMEMPENHO"),  
    col("VALOR"),  
    col("DATA"),  
    lit(None).alias("NUMPROCESSO")  
)
```

3. Liq_M: Filtrando movimentos 6 e 7 e concatenando TIPO_DUEOF e TIPOMOVIMENTO

```
df_liq_M_ready = df_liq_M.select(  
    concat(col("TIPO_DUEOF"), col("TIPOMOVIMENTO")).alias("TIPO_DUEOF"),  
    col("NUMEMPENHO"),
```

```
col("VALOR"),
col("DATA"),
lit(None).alias("NUMPROCESSO")
).filter(col("TIPOMOVIMENTO").isin("6", "7"))
```

4. CONSOLIDAÇÃO FINAL (Union das 3 fontes)

```
df_final_linhas = df_dueof_ready \
    .union(df_liq_D_ready) \
    .union(df_liq_M_ready)
```

5. --- ETAPA DE ETL: CORREÇÃO DE TIPOS ---

```
df_final_linhas = df_final_linhas.withColumn(
    # lpad garante que datas como 02012026 não percam o '0' à esquerda se forem tratadas
    # como número
    "DATA", to_date(lpad(col("DATA").cast("string"), 8, '0'), "ddMMyyyy")
).withColumn(
    # DecimalType(18, 2) resolve o problema das casas decimais "pulando" no Excel
    "VALOR", col("VALOR").cast(DecimalType(18, 2))
)
```

6. Ordenação Lógica

```
df_final_linhas = df_final_linhas.orderBy("NUMEMPENHO", "DATA")
```

Exibição do resultado

```
display(df_final_linhas.limit(5))
```

COMMAND -----

```

#DATAFRAME DOTACAO AGREGADO (df_dotacao)

dotacao_subtrai = ["02", "13", "33", "36", "38", "45"]

dotacao_soma = ["01", "12", "25", "34", "35", "37", "32", "46"]


df_dotacao = df_final_linhas \

.withColumn("NUMDOTACAO", substring(col("NUMEMPENHO"), 1, 11)) \

.groupBy("NUMDOTACAO") \

.agg(

    sum(when(col("TIPO_DUEOF") == "32",
col("VALOR")).otherwise(0)).alias("DOTACAO_INICIAL"),

    (

        #sum(when(col("TIPO_DUEOF").isin(array(*dotacao_soma))

        sum(when(col("TIPO_DUEOF").isin(dotacao_soma), col("VALOR")).otherwise(0)) -

        #sum(when(col("TIPO_DUEOF").isin(array(*dotacao_subtrai))

        sum(when(col("TIPO_DUEOF").isin(dotacao_subtrai), col("VALOR")).otherwise(0))

    ).alias("DOTACAO_ATUALIZADA"),

    (

        sum(when(col("TIPO_DUEOF") == "03", col("VALOR")).otherwise(0)) +

        sum(when(col("TIPO_DUEOF") == "31", col("VALOR")).otherwise(0)) +

        sum(when(col("TIPO_DUEOF") == "07", col("VALOR")).otherwise(0)) -

        sum(when(col("TIPO_DUEOF") == "04", col("VALOR")).otherwise(0)) -

        sum(when(col("TIPO_DUEOF") == "30", col("VALOR")).otherwise(0))

    ).alias("DOTACAO_EMPENHADA"), #todos empenhos da dotação

    ((

```

```

sum(when(col("TIPO_DUEOF").isin(dotacao_soma), col("VALOR")).otherwise(0)) -
sum(when(col("TIPO_DUEOF").isin(dotacao_subtrai), col("VALOR")).otherwise(0))
) -
(
sum(when(col("TIPO_DUEOF") == "03", col("VALOR")).otherwise(0)) +
sum(when(col("TIPO_DUEOF") == "31", col("VALOR")).otherwise(0)) +
sum(when(col("TIPO_DUEOF") == "07", col("VALOR")).otherwise(0)) -
sum(when(col("TIPO_DUEOF") == "04", col("VALOR")).otherwise(0)) -
sum(when(col("TIPO_DUEOF") == "30", col("VALOR")).otherwise(0))
)).alias("DOTACAO_A_EMPENHAR"),
min(when(col("TIPO_DUEOF") == "32", col("DATA"))).alias("DATA")
)

display(df_dotacao.limit(5))

# COMMAND -----

# 1. Primeiro, criamos a agregação por NUMEMPENHO (os valores específicos do empenho)
df_empenho_agregado = df_final_linhas.groupBy("NUMEMPENHO").agg(
(
sum(when(col("TIPO_DUEOF") == "05", col("VALOR")).otherwise(0)) +
sum(when(col("TIPO_DUEOF") == "27", col("VALOR")).otherwise(0)) +
sum(when(col("TIPO_DUEOF") == "08", col("VALOR")).otherwise(0)) -
sum(when(col("TIPO_DUEOF") == "26", col("VALOR")).otherwise(0)) -
sum(when(col("TIPO_DUEOF") == "06", col("VALOR")).otherwise(0))
).alias("LIQUIDACAO_PAGO"), #saldo de ordem de pagamento

```

```

(
  sum(when(col("TIPO_DUEOF") == "D", col("VALOR")).otherwise(0)) +
  sum(when(col("TIPO_DUEOF") == "M6", col("VALOR")).otherwise(0)) -
  sum(when(col("TIPO_DUEOF") == "M7", col("VALOR")).otherwise(0))
).alias("LIQUIDADO"), #saldo de liquidação

(
  sum(when(col("TIPO_DUEOF") == "03", col("VALOR")).otherwise(0))
).alias("EMPENHO_INICIAL"), #primeiro empenho

(
  sum(when(col("TIPO_DUEOF") == "03", col("VALOR")).otherwise(0)) +
  sum(when(col("TIPO_DUEOF") == "31", col("VALOR")).otherwise(0)) +
  sum(when(col("TIPO_DUEOF") == "07", col("VALOR")).otherwise(0)) -
  sum(when(col("TIPO_DUEOF") == "04", col("VALOR")).otherwise(0)) -
  sum(when(col("TIPO_DUEOF") == "30", col("VALOR")).otherwise(0))
).alias("EMPENHO_ATUALIZADO"), #empenho atualizado

((
  sum(when(col("TIPO_DUEOF") == "D", col("VALOR")).otherwise(0)) +
  sum(when(col("TIPO_DUEOF") == "M6", col("VALOR")).otherwise(0)) -
  sum(when(col("TIPO_DUEOF") == "M7", col("VALOR")).otherwise(0))
) - (
  sum(when(col("TIPO_DUEOF") == "05", col("VALOR")).otherwise(0)) +
  sum(when(col("TIPO_DUEOF") == "27", col("VALOR")).otherwise(0)) +
  sum(when(col("TIPO_DUEOF") == "08", col("VALOR")).otherwise(0)) -
  sum(when(col("TIPO_DUEOF") == "26", col("VALOR")).otherwise(0)) -
  sum(when(col("TIPO_DUEOF") == "06", col("VALOR")).otherwise(0))
)).alias("LIQUIDACAO_A_PAGAR"), # LIQUIDADO - LIQUIDACAO_PAGO

```

```
((
  sum(when(col("TIPO_DUEOF") == "03", col("VALOR")).otherwise(0)) +
  sum(when(col("TIPO_DUEOF") == "31", col("VALOR")).otherwise(0)) +
  sum(when(col("TIPO_DUEOF") == "07", col("VALOR")).otherwise(0)) -
  sum(when(col("TIPO_DUEOF") == "04", col("VALOR")).otherwise(0)) -
  sum(when(col("TIPO_DUEOF") == "30", col("VALOR")).otherwise(0))
)-{
  sum(when(col("TIPO_DUEOF") == "D", col("VALOR")).otherwise(0)) +
  sum(when(col("TIPO_DUEOF") == "M6", col("VALOR")).otherwise(0)) -
  sum(when(col("TIPO_DUEOF") == "M7", col("VALOR")).otherwise(0))
}).alias("EMPENHO_A_LIQUIDAR") # EMPENHO_ATUALIZADO - LIQUIDADO
)
```

2. Criamos a chave de ligação (11 dígitos) no dataframe de empenhos

```
df_empenho_com_chave = df_empenho_agregado.withColumn("CHAVE_DOTACAO",
  substring(col("NUMEMPENHO"), 1, 11))
```

3. O JOIN Mágico: Cruzamos os empenhos com o df_dotacao que você já criou

```
df_relatorio_final = df_empenho_com_chave.join(
  df_dotacao,
  df_empenho_com_chave.CHAVE_DOTACAO == df_dotacao.NUMDOTACAO,
  "left"
)
```

4. Seleção final das colunas para ficar limpo


```
df_final = df_relatorio_final.select(
    "NUMEMPENHO",
    "NUMDOTACAO",
    "DOTACAO_INICIAL",
    "DOTACAO_ATUALIZADA",
    "DOTACAO_EMPENHADA",
    "DOTACAO_A_EMPENHAR",
    "EMPENHO_INICIAL",
    "EMPENHO_ATUALIZADO",
    "EMPENHO_A_LIQUIDAR",
    "LIQUIDADO",
    "LIQUIDACAO_PAGO",
    "LIQUIDACAO_A_PAGAR"
).orderBy("NUMEMPENHO")
```

```
# Visualização
```

```
display(df_final.limit(10))
```

```
# COMMAND -----
```

```
# EXPORTAÇÃO DAS TABELAS EM CSV
```

```
# 1. Configurações de Caminho e Credencial Global
```

```
container_silver = "silver-dados-processados"
```

```
base_path =
```

```
f"wasbs://{container_silver}@{storage_account_name}.blob.core.windows.net/tabelas_csv"
```

```
# ISSO AQUI É A CHAVE: Define a permissão para TODO o notebook de uma vez
spark.conf.set(
    f"fs.azure.account.key.{storage_account_name}.blob.core.windows.net",
    storage_account_key
)
```

```
# 2. DEFINA OS NOMES PERSONALIZADOS
```

```
export_map = {
    "df_final_linhas": "base_eventos_linhas",
    "df_dotacao": "dotacao_consolidada",
    "df_final": "painel_empenhos_consolidado"
}
```

```
print("Iniciando exportação com nomes fixos e credenciais globais...")
```

```
for df_origem, nome_personalizado in export_map.items():
    temp_folder = f"{base_path}/{nome_personalizado}_temp"
    final_file_path = f"{base_path}/{nome_personalizado}.csv"
```

```
# Pega o DataFrame real pelo nome
df_para_gravar = globals()[df_origem]
```

```
print(f"Gerando arquivo: {nome_personalizado}.csv")
```

```
# Etapa 1: Grava como pasta (agora a credencial já está no sistema)
```

```
df_para_gravar.coalesce(1).write.format("csv") \
```

```
.mode("overwrite") \
.option("header", "true") \
.option("delimiter", ";") \
.save(temp_folder)

# Etapa 2: Localiza o arquivo e renomeia
# Agora o dbutils terá acesso porque configuramos o spark.conf.set acima
files = dbutils.fs.ls(temp_folder)
csv_temp_file = [f.path for f in files if f.path.endswith(".csv")][0]

dbutils.fs.cp(csv_temp_file, final_file_path)
dbutils.fs.rm(temp_folder, recurse=True)

print("--- Sucesso! Arquivos nomeados corretamente no Azure. ---")
```